



INTERVIEW QUESTIONS



The Key Concepts To Master

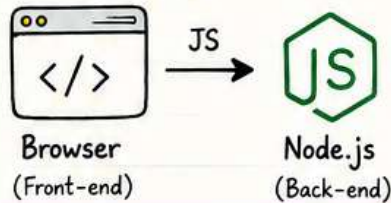


Node.js is a JavaScript runtime built on Chrome's V8 engine. It helps us build **fast**, **scalable** & **real-time** backend applications.

1 Q. What is Node.js?

Ans: Node.js is a JavaScript runtime environment that allows us to execute JavaScript code outside the browser. It is widely used to build **scalable** and **high-performance** backend applications.

Think of it as:



Why Learn Node.js?

- ✓ Fast & Scalable
- ✓ Handles many connections efficiently
- ✓ Great for Real-time apps
- ✓ Huge Ecosystem (npm)
- ✓ High Demand in Industry

2 Q. What is the difference between Node.js and JavaScript?

Ans: JavaScript is a programming language, while Node.js is a runtime environment that allows JavaScript to run on the server.

	JavaScript	Node.js
</>	Programming Language	Runtime Environment
🌐	Runs in Browser	Runs on Server (Outside Browser)
👤	Used for Front-end	Used for Back-end
⚙️	Needs Runtime (Browser)	Built on Chrome's V8 Engine

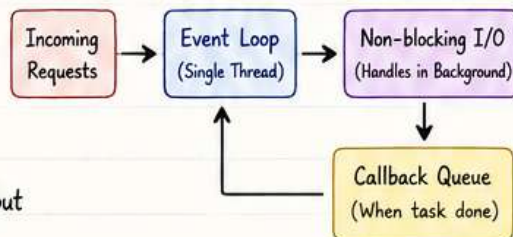
3 Q. Is Node.js single-threaded?

Ans: Yes, Node.js is **single-threaded**, but it uses an **event-driven, non-blocking I/O** model to handle multiple requests efficiently.



Memory Trick: Node.js is "**Single Threaded**", but **Multi-capable**" because of Event Loop!

How it works?



Top Use Cases

- ★ Web Servers
- ★ REST APIs
- ★ Real-time Chat Apps
- ★ Streaming Apps
- ★ Microservices
- ★ Tools & CLIs
- and many more...

4 Q. What kind of API function is supported by Node.js?

Ans: Node.js supports both **Synchronous** (blocking) and **Asynchronous** (non-blocking) API functions.

Synchronous (Blocking)

- Executes line by line
- Each task waits for the previous one to complete
- Easier to write but less efficient

Example: `fs.readFileSync()`

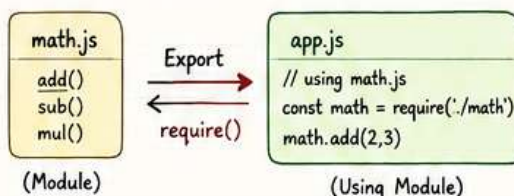
Asynchronous (Non-blocking)

- Does not wait for the task to complete
- Better performance & handles many requests
- Uses Callbacks, Promises, or Async/Await

Example: `fs.readFile()`

5 Q. What is a module in Node.js?

Ans: A module is a **reusable block of code** that can be **exported** and **reused** in different parts of an application.



Focus on concepts, practice daily & build projects.



SAVE
this post
for later



SHARE
it with your
friends



COMMENT
your doubts
below

FOLLOW
THE BINARY TECHNOLOGY
for more tech content



INTERVIEW QUESTIONS

2/4

The Key Concepts To Master



6. What is npm and its advantages?

Ans: npm (Node Package Manager) is the default package manager for Node.js. It helps in managing packages (dependencies), versions and sharing code.

npm Advantages



Dependency Management
Easily install, update and remove packages.



Version Control
Manage different versions of packages smoothly.



Centralized Repository
Huge collection of open-source packages.



Quick Tip

Most JS libraries you use daily (like express, mongoose) come from **npm**!

7. What is middleware?

Ans: Middleware functions are functions that run between the request and response cycle.

They can modify the request, response, end the request-response cycle, or call the next middleware.

Middleware Flow



Used for: Logging, Authentication, Validation, Error handling, Data parsing, etc.

Example Use

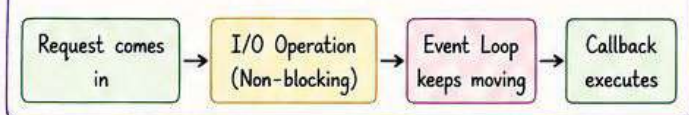
- ★ Logger
- ★ Auth Check
- ★ Body Parser
- ★ Error Handler

8. How does Node.js handle concurrency despite being single-threaded?

Ans: Node.js uses non-blocking I/O operations and the event loop to handle many requests concurrently.

It offloads heavy operations to the system kernel and processes callbacks when ready.

How It Works?



Think of it like:

You order food (request), chef (I/O) works on it in background, you keep doing other things, chef calls you when its ready (callback).

9. What is control flow in Node.js?

Ans: Control flow is the order in which code is executed. In Node.js, it is mainly about handling asynchronous operations, callbacks, promises, and errors in the correct sequence.

Control Flow Example

readFile() (Async) → Callback executes → Next operation

Proper control flow = Efficient, bug-free code

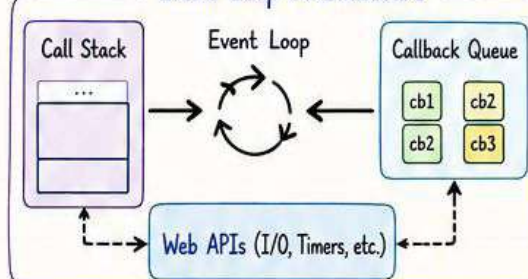
★ Interview Note

Always handle errors in async code using try/catch, .catch() or error-first callbacks.

10. What do you mean by event loop in Node.js?

Ans: The event loop is the heart of Node.js. It continuously checks the call stack, callback queue, and other queues to execute callbacks. This allows Node.js to perform non-blocking, asynchronous operations.

Event Loop Architecture



Memory Trick

Event Loop keeps the loop running, so your app never blocks!



SAVE
this post
for later



SHARE
it with your
friends



COMMENT
your doubts
below

FOLLOW
The Binary Technology
for more tech content



INTERVIEW QUESTIONS

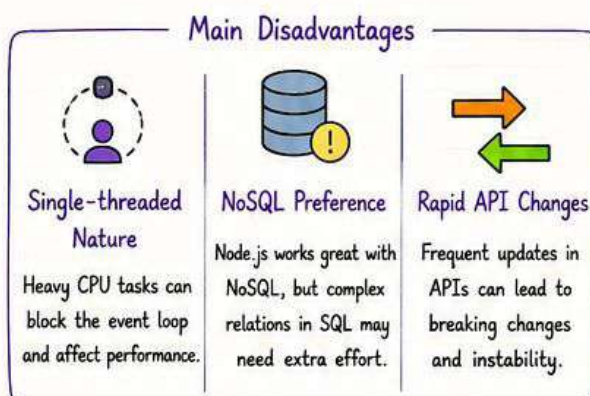
3/4

The Key Concepts To Master



11 Q. What are the main disadvantages of Node.js?

Ans: Like every technology, Node.js has a few **limitations** you should know.



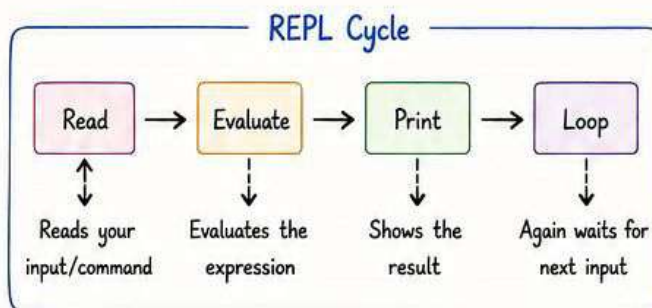
Interview Tip

Always mention real-world trade-offs instead of just pointing out negatives.

12 Q. What is REPL in Node.js?

Ans: REPL stands for **Read, Evaluate, Print, Loop**.

It is an interactive shell that allows you to write and execute Node.js code line by line. Great for testing and debugging small pieces of code quickly.



Try it!

Open terminal and type:

```
node
```

You are in REPL mode! 🚀

13 Q. How to import a module in Node.js?

Ans: We use the **require()** function to import built-in or external modules into our application.

```
// Importing built-in module
const fs = require('fs');

// Importing external module
const express = require('express');

// Now you can use it
fs.readFile('file.txt', 'utf8', (err, data) => {
  console.log(data);
});
```

★ Key Point

require() is a synchronous call but modules are cached after the first load for better performance.

14 Q. What is the difference between Node.js and AJAX?

Ans: They are used on different sides and serve different purposes.

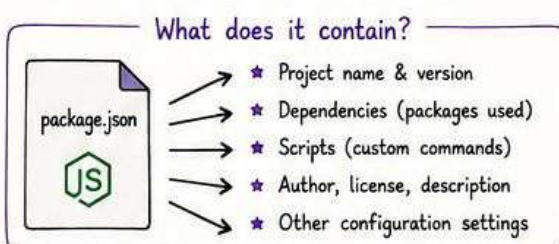
	Node.js	vs	AJAX
Type	Server-side runtime environment		Client-side technique
Purpose	Build scalable backend applications		Update parts of a web page without reloading
Language	JavaScript (on Server)		JavaScript (in Browser)
Example	REST API, Web Server, CLI tools		Live search, form validation

✓ Remember

Node.js runs on the server, AJAX runs in the browser!

15 Q. What is package.json in Node.js?

Ans: package.json is a **metadata file** that contains important information about your project.



🚀 Pro Tip

Use npm init in terminal to create it easily.



SAVE
this post
for later



SHARE
it with your
friends



COMMENT
your doubts
below

FOLLOW
The Binary Technology
for more tech content



INTERVIEW QUESTIONS

The Key Concepts To Master



16 Q. What is the most popular Node.js framework used these days?

Ans: The most popular Node.js framework is **Express.js**. It is fast, minimal and flexible, making it perfect for building web applications and APIs.

Why Express.js?

- ✓ Minimal and lightweight
- ✓ Easy routing and middleware support
- ✓ High performance
- ✓ Huge community support
- ✓ Scalable for small to enterprise apps

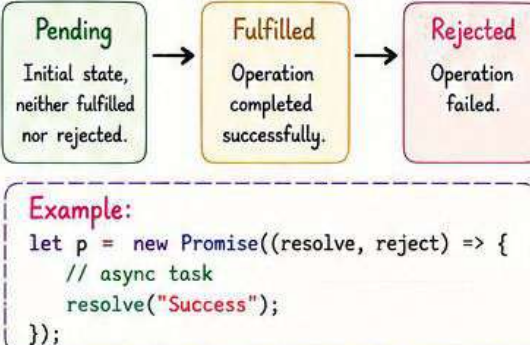
Real World Use

Apps like Uber, Paytm, and MyFitnessPal use Express.js in their stack!

17 Q. What are promises in Node.js?

Ans: Promises are objects that represent the **eventual completion** (or failure) of an asynchronous operation. They help avoid callback hell and make code cleaner and easier to read.

Promise Life Cycle



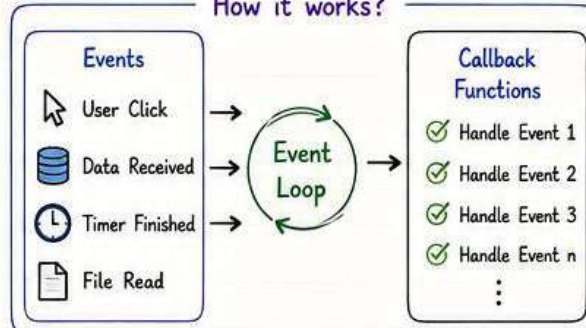
Topper Tip

Prefer Promises or async/await over callbacks for better readability!

18 Q. What is event-driven programming in Node.js?

Ans: Event-driven programming means the flow of the program is controlled by **events** (like user actions, I/O operations, timers etc). Node.js uses an **Event Loop** to listen for events and trigger callbacks.

How it works?



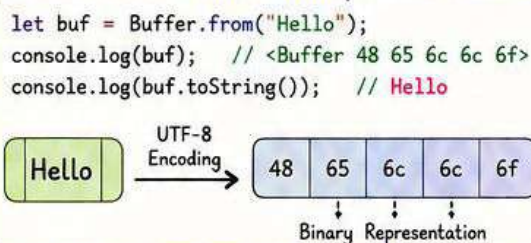
Remember

Node.js = Events + Event Loop + Callbacks = Non-blocking Magic ✨

19 Q. What is buffer in Node.js?

Ans: A Buffer is a temporary storage area used to store **binary data**. It is mainly used while working with streams, files, TCP packets, images, audio, etc.

Buffer Example



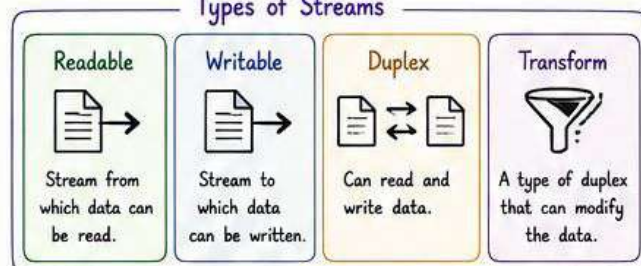
Key Point

Buffer lets Node.js handle raw binary data efficiently at low level!

20 Q. What are streams in Node.js?

Ans: Streams are objects that allow you to read data from a source or write data to a destination **continuously**. They are memory-efficient and are used for handling large amounts of data.

Types of Streams



Pro Tip

Use streams instead of reading entire files into memory!



SAVE
this post
for later



SHARE
it with your
friends



COMMENT
your doubts
below

FOLLOW
The Binary Technology
for more tech content

